

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №1

По дисциплине «Интеллектуальный анализ данных»

Тема: «РСА»

Выполнил:

Студент 4 курса

Группы ИИ-26

Кушнеревич Е.А

Проверил:

Никитин С. К.

Брест 2026

Цель работы: Научиться применять метод PCA для осуществления визуализации данных

Ход работы:

1. Используя выборку по варианту, осуществить проецирование данных на плоскость первых двух и трех главных компонент (двумя способами: 1. вручную через использование `numpy.linalg.eig` для вычисления собственных значений и собственных векторов и 2. с помощью `sklearn.decomposition.PCA` для непосредственного применения метода PCA – два независимых варианта решения);
2. Выполнить визуализацию полученных главных компонент с использованием средств библиотеки `matplotlib`, обозначая экземпляры разных классов с использованием разных цветовых маркеров;
3. Используя собственные значения, рассчитанные на этапе 1, вычислить потери, связанные с преобразованием по методу PCA. Сделать выводы;
4. Оформить отчет по выполненной работе, загрузить исходный код и отчет в соответствующий репозиторий на github.

№ варианта	Выборка	Класс
8	xasens.zip	Diagnosis ID

Код программы:

```
import numpy as np
import pandas as pd
import matplotlib
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer

RAW_PATH = r"D:\exasens\Exasens.csv"

def load_data(path: str) -> pd.DataFrame:
    df = pd.read_csv(path, skiprows=[1, 2], header=0)
    df = df.iloc[:, :9]
    df.columns = [
        "Diagnosis", "ID", "Imag_Min", "Imag_Avg",
        "Real_Min", "Real_Avg", "Gender", "Age", "Smoking",
    ]
    return df

df = load_data(RAW_PATH)

FEATURE_COLS = ["Imag_Min", "Imag_Avg", "Real_Min", "Real_Avg",
                "Gender", "Age", "Smoking"]
X_raw = df[FEATURE_COLS].values.astype(float)
y = df["Diagnosis"].values
```

```

print("Пропуски по признакам (до замещения):")
print(pd.DataFrame(X_raw, columns=FEATURE_COLS).isna().sum().to_string())

imputer = SimpleImputer(strategy="median")
X_imputed = imputer.fit_transform(X_raw)

print(f"\nРазмер выборки после замещения пропусков: {X_imputed.shape[0]} "
      f"объектов, {X_imputed.shape[1]} признаков")
print(df["Diagnosis"].value_counts().to_string())
print()

scaler = StandardScaler()
X = scaler.fit_transform(X_imputed)

classes = ["HC", "Asthma", "Infected", "COPD"]
colors = {"HC": "tab:green", "Asthma": "tab:orange",
          "Infected": "tab:blue", "COPD": "tab:red"}
markers = {"HC": "o", "Asthma": "^", "Infected": "s", "COPD": "D"}

def pca_manual(data: np.ndarray, n_components: int):
    cov_matrix = np.cov(data.T)
    eigvals, eigvecs = np.linalg.eig(cov_matrix)
    eigvals = eigvals.real
    eigvecs = eigvecs.real

    order = np.argsort(-eigvals)
    eigvals_sorted = eigvals[order]
    eigvecs_sorted = eigvecs[:, order]

    W = eigvecs_sorted[:, :n_components]
    projected = data.dot(W)
    return projected, eigvals_sorted, eigvecs_sorted

X_manual_2d, eigvals_manual, eigvecs_manual = pca_manual(X, 2)
X_manual_3d, _, _ = pca_manual(X, 3)

pca2 = PCA(n_components=2)
X_sklearn_2d = pca2.fit_transform(X)

pca3 = PCA(n_components=3)
X_sklearn_3d = pca3.fit_transform(X)

def scatter_2d(ax, data, labels, title):
    for c in classes:
        mask = labels == c
        ax.scatter(data[mask, 0], data[mask, 1], label=c,
                  color=colors[c], marker=markers[c],
                  alpha=0.75, edgecolor="k", s=45)
    ax.set_xlabel("PC1")
    ax.set_ylabel("PC2")
    ax.set_title(title)
    ax.legend()
    ax.grid(alpha=0.3)

fig, axes = plt.subplots(1, 2, figsize=(13, 5.5))
scatter_2d(axes[0], X_manual_2d, y, "PCA вручную")
scatter_2d(axes[1], X_sklearn_2d, y, "PCA через sklearn")
plt.suptitle("Проекция на первые 2 главные компоненты", fontsize=13)
plt.tight_layout()
plt.savefig("pca_2d_comparison.png", dpi=150)

```

```

plt.show()
plt.close()

def scatter_3d(ax, data, labels, title):
    for c in classes:
        mask = labels == c
        ax.scatter(data[mask, 0], data[mask, 1], data[mask, 2], label=c,
                   color=colors[c], marker=markers[c],
                   alpha=0.75, edgecolor="k", s=45)
    ax.set_xlabel("PC1")
    ax.set_ylabel("PC2")
    ax.set_zlabel("PC3")
    ax.set_title(title)
    ax.legend()

fig = plt.figure(figsize=(13, 6))
ax1 = fig.add_subplot(1, 2, 1, projection="3d")
scatter_3d(ax1, X_manual_3d, y, "PCA вручную (3 компоненты)")
ax2 = fig.add_subplot(1, 2, 2, projection="3d")
scatter_3d(ax2, X_sklearn_3d, y, "PCA через sklearn (3 компоненты)")
plt.suptitle("Проекция на первые 3 главные компоненты", fontsize=13)
plt.tight_layout()
plt.savefig("pca_3d_comparison.png", dpi=150)
plt.show()
plt.close()

full_info = eigvals_manual.sum()
loss_2 = 100 - eigvals_manual[:2].sum() / full_info * 100
loss_3 = 100 - eigvals_manual[:3].sum() / full_info * 100

print("Собственные значения ковариационной матрицы (по убыванию):")
for i, v in enumerate(eigvals_manual, 1):
    print(f" lambda_{i} = {v:.4f}   доля объясненной дисперсии = "
          f"{v / full_info * 100:5.2f}%")

print(f"\nСуммарная дисперсия: {full_info:.4f}")
print(f"Потери информативности при сведении к 2 компонентам: {loss_2:.2f}%")
print(f"Потери информативности при сведении к 3 компонентам: {loss_3:.2f}%")

print("\nПроверка через sklearn:")
print(" 2 компоненты:", np.round(pca2.explained_variance_ratio_, 4),
      " сумма =", round(pca2.explained_variance_ratio_.sum() * 100, 2), "%")
print(" 3 компоненты:", np.round(pca3.explained_variance_ratio_, 4),
      " сумма =", round(pca3.explained_variance_ratio_.sum() * 100, 2), "%")

```

Результат программы:

Пропуски по признакам (до замещения):

```

Imag_Min  299
Imag_Avg  299
Real_Min   299
Real_Avg   299
Gender      0
Age         0
Smoking     0

```

Размер выборки после замещения пропусков: 399 объектов, 7 признаков

Diagnosis

```

HC      160

```

Asthma 80
 Infected 80
 COPD 79

Собственные значения ковариационной матрицы (по убыванию):

lambda_1 = 2.7784 доля объясненной дисперсии = 39.59%
 lambda_2 = 1.3302 доля объясненной дисперсии = 18.96%
 lambda_3 = 1.1196 доля объясненной дисперсии = 15.95%
 lambda_4 = 0.8819 доля объясненной дисперсии = 12.57%
 lambda_5 = 0.7597 доля объясненной дисперсии = 10.83%
 lambda_6 = 0.1385 доля объясненной дисперсии = 1.97%
 lambda_7 = 0.0091 доля объясненной дисперсии = 0.13%

Суммарная дисперсия: 7.0176

Потери информативности при сведении к 2 компонентам: 41.45%

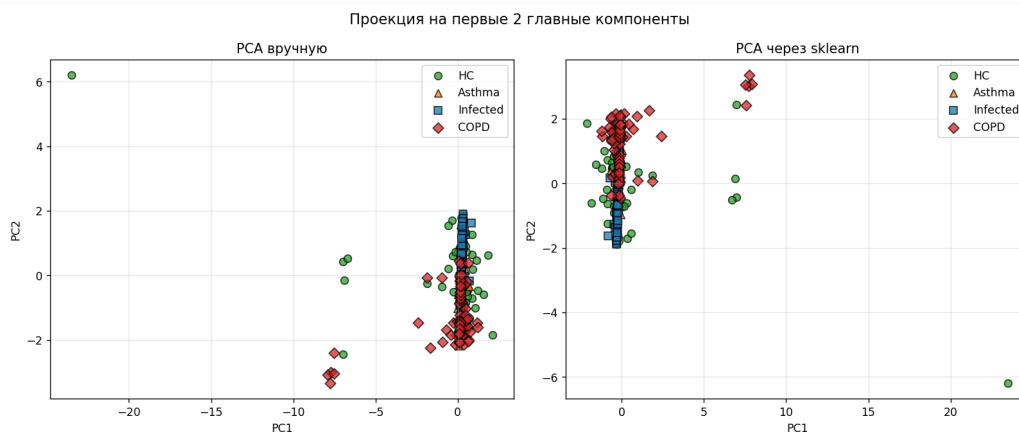
Потери информативности при сведении к 3 компонентам: 25.50%

Проверка через sklearn:

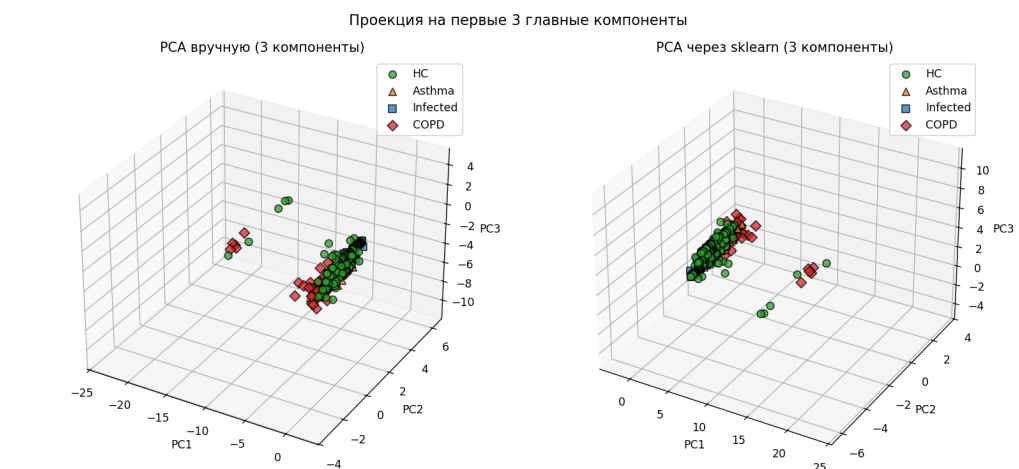
2 компоненты: [0.3959 0.1896] сумма = 58.55 %

3 компоненты: [0.3959 0.1896 0.1595] сумма = 74.5 %

Проекция 2-х компонентов:



Проекция 3-х компонентов:



Вывод: Прodelав данную лабораторную работу я научился применять метод PCA для осуществления визуализации данных